

CS202: Software Tools and Techniques for CSE

Lecture 5

Shouvick Mondal

shouvick.mondal@iitgn.ac.in
January 2025



Program analysis

Contents

hide

Article Talk

Read Edit View history Tools ▼

From Wikipedia, the free encyclopedia

For other uses, see [Program analysis \(disambiguation\)](#).



This article **needs additional citations for verification**. Please help [improve this article](#) by [adding citations to reliable sources](#). Unsourced material may be challenged and removed.
Find sources: "Program analysis" – news · newspapers · books · scholar · JSTOR (February 2018) ([Learn how and when to remove this message](#))

In [computer science](#), **program analysis**^[1] is the process of analyzing the behavior of computer programs regarding a property such as correctness, robustness, safety and liveness. Program analysis focuses on two major areas: [program optimization](#) and [program correctness](#). The first focuses on improving the program's performance while reducing the resource usage while the latter focuses on ensuring that the program does what it is supposed to do.

Program analysis can be performed without executing the program ([static program analysis](#)), during runtime ([dynamic program analysis](#)) or in a combination of both.

Part of a series on

Software development

Core activities [\[show\]](#)

Paradigms and models [\[show\]](#)

Methodologies and frameworks [\[show\]](#)

Supporting disciplines [\[show\]](#)

Practices [\[show\]](#)

Tools [\[show\]](#)



What is the role of a software tool here? Is there any...?

What is Program Analysis?

For an end-goal, identify “interesting aspects” of a program's representation.

What is Program Analysis?

For an end-goal,
identify “interesting aspects” of
a program's representation.

Checking security

Array index range

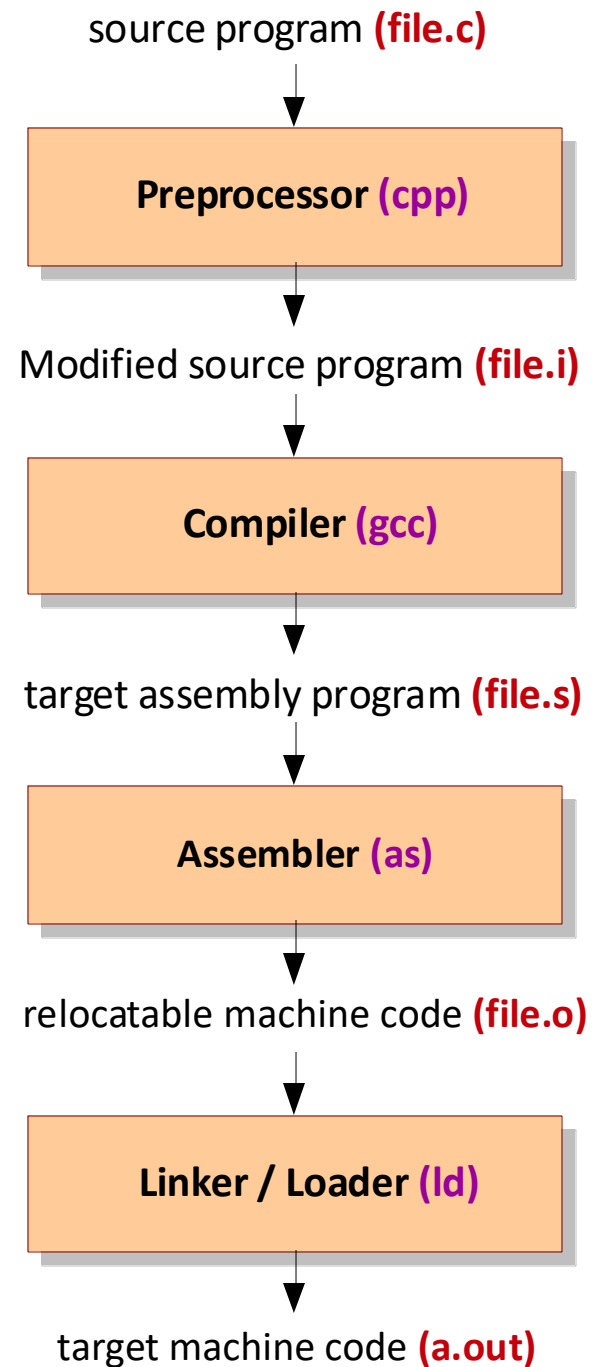
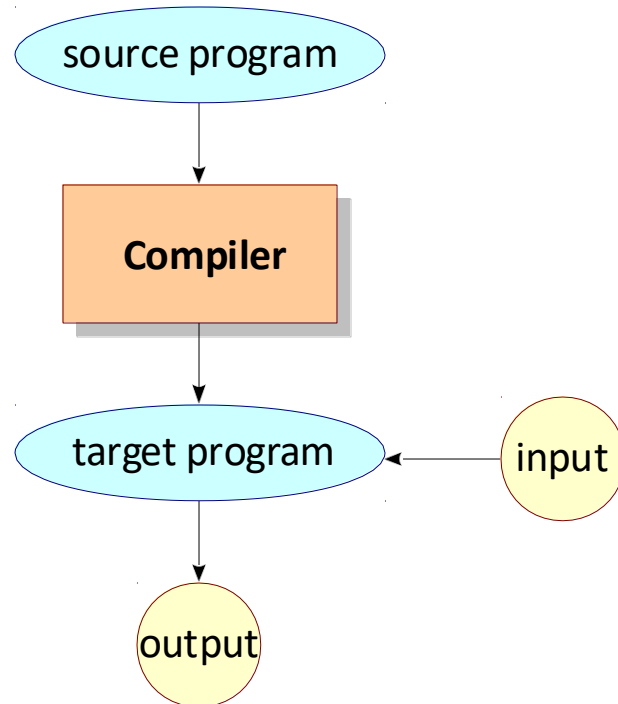
Source, AST, binary,
executed instruction

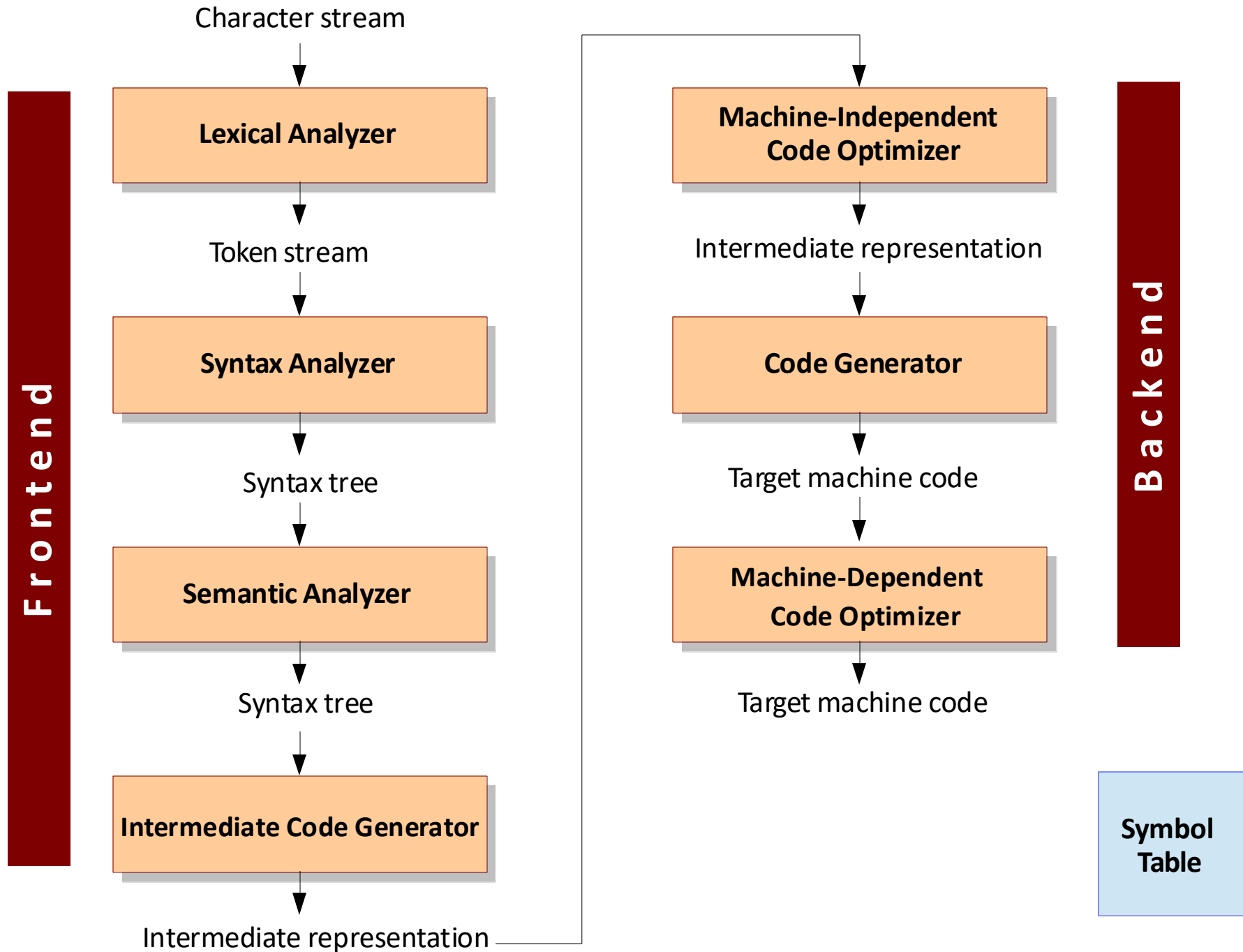
Examples

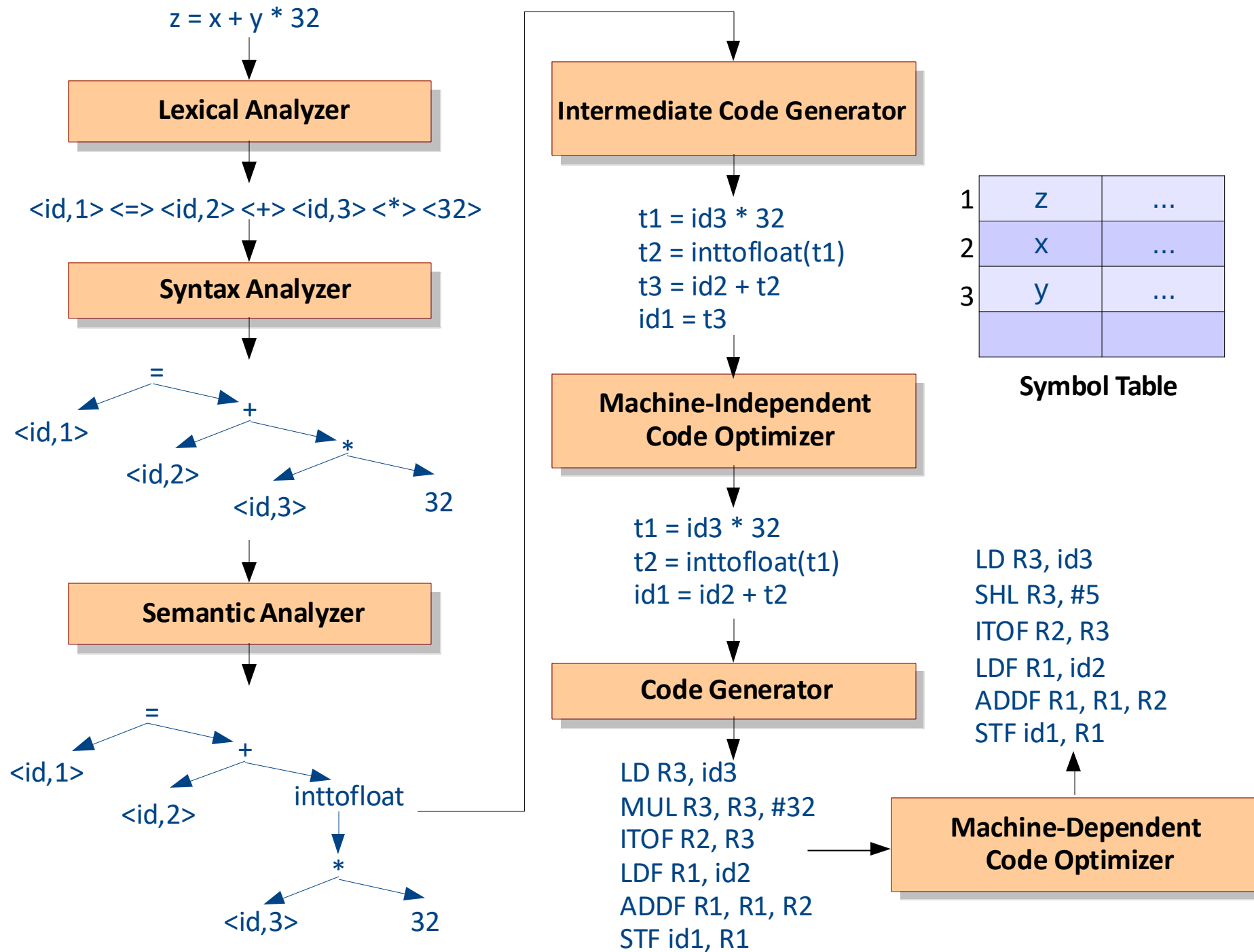
End goal	Interesting aspect
Dead code elimination	Reachability
Constant propagation	use-def
Security	Array index range, dangling pointers
Parallelization	Data dependence, SIMD opportunities
Debugging	Slice
Cache performance	Memory access pattern
Memory reduction	Live ranges
...	...

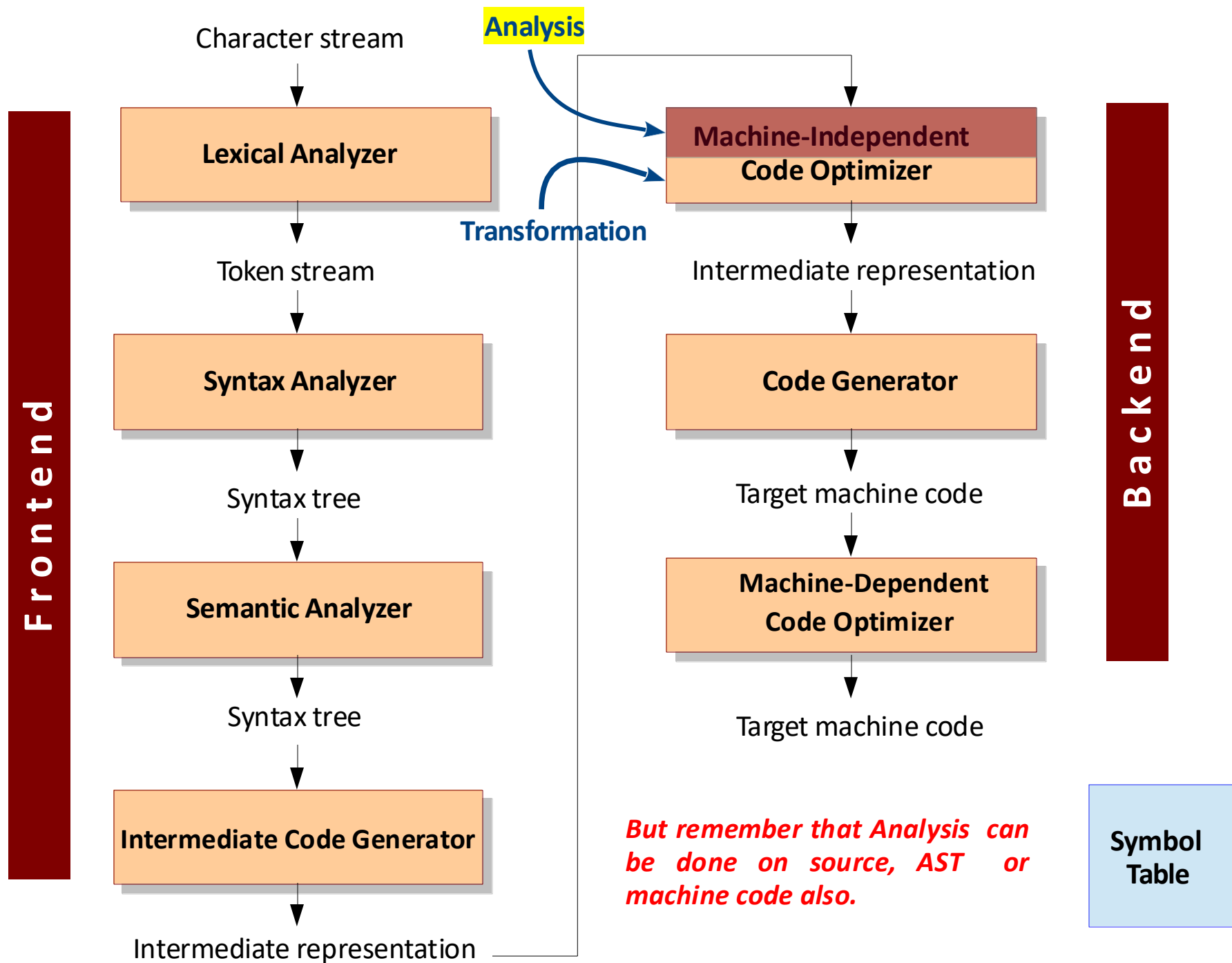
Program Analysis is often a pre-cursor to Optimization.

```
:~$ gcc file.c  
:~$ ./a.out
```









Static Program Analysis

```
//filename: fact.c
#include <stdio.h>
int factorial(int n)
{
    if (n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}
int main()
{
    int n;
    printf("Enter a non-negative integer: ");
    scanf("%d", &n);
    printf("The factorial of %d is %d.\n", n, factorial(n));
    return 0;
}
```

Show the function call graph of the C code in textual mode and graphical mode.

```
//filename: fact.c
#include <stdio.h>
int factorial(int n)
{
    if (n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}
int main()
{
    int n;
    printf("Enter a non-negative integer: ");
    scanf("%d", &n);
    printf("The factorial of %d is %d.\n", n, factorial(n));
    return 0;
}
```

[**Textual mode**] (generate function call graph)

cflow fact.c

main() <int main () at fact.c:9>:

printf()

scanf()

factorial() <int factorial (int n) at fact.c:2> (R):

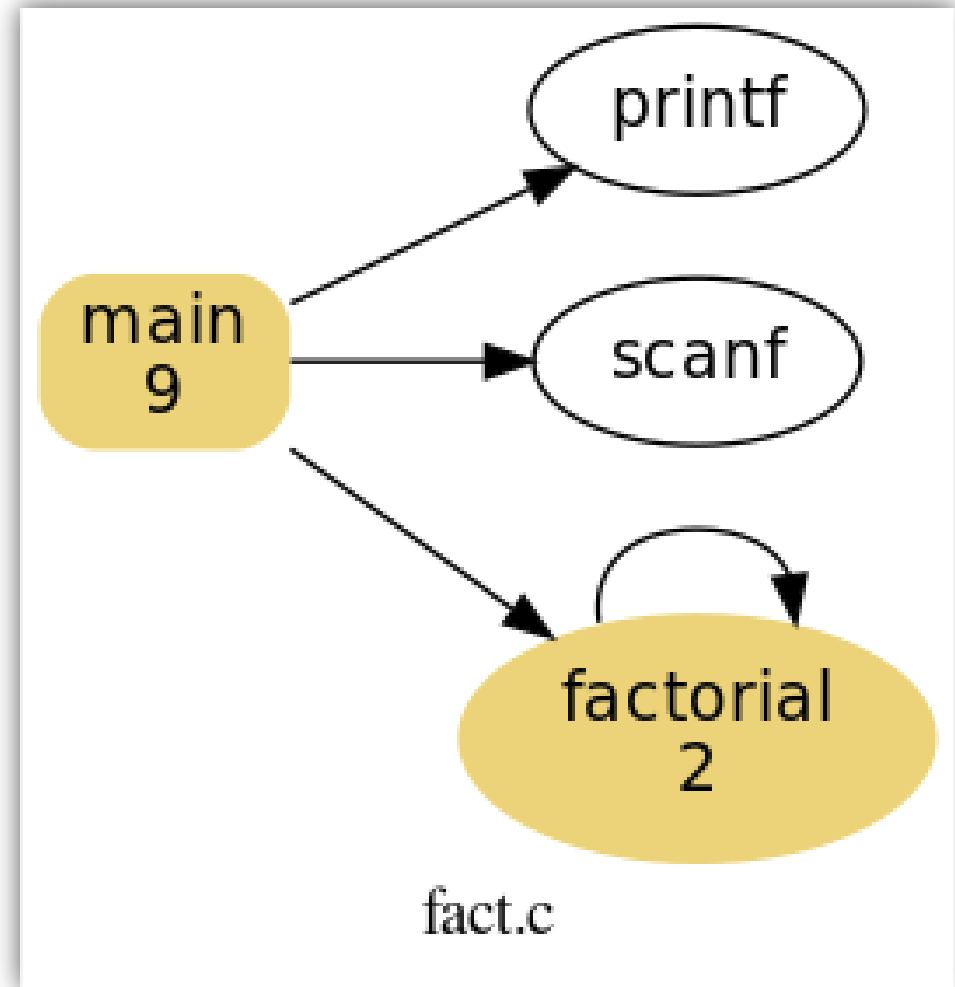
factorial() <int factorial (int n) at fact.c:2> (recursive: see 4)

```
//filename: fact.c
#include <stdio.h>
int factorial(int n)
{
    if (n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}
int main()
{
    int n;
    printf("Enter a non-negative integer: ");
    scanf("%d", &n);
    printf("The factorial of %d is %d.\n", n, factorial(n));
    return 0;
}
```

[**Graphical mode**] (generate function call graph)

cflow2dot -i fact.c

found cflow at: /usr/bin/cflow
 found dot at: /usr/bin/dot
 This may take some time...
 svg produced successfully from dot.



Static Program Analysis

```
//filename: fact.c
#include <stdio.h>
int factorial(int n)
{
    if (n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}
int main()
{
    int n;
    printf("Enter a non-negative integer: ");
    scanf("%d", &n);
    printf("The factorial of %d is %d.\n", n, factorial(n));
    return 0;
}
```

Show the control flow graph of the C code.



```
//fact.c
#include <stdio.h>
int factorial(int n)
{
    if (n == 0)
```

Step 1[Generate control flow graph]

gcc -fdump-tree-cfg-graph-lineno fact.c

```
    return n * factorial(n - 1);
}
int main()
{
    int n;
```



a.out



fact.c



fact.c.012t.
cfg



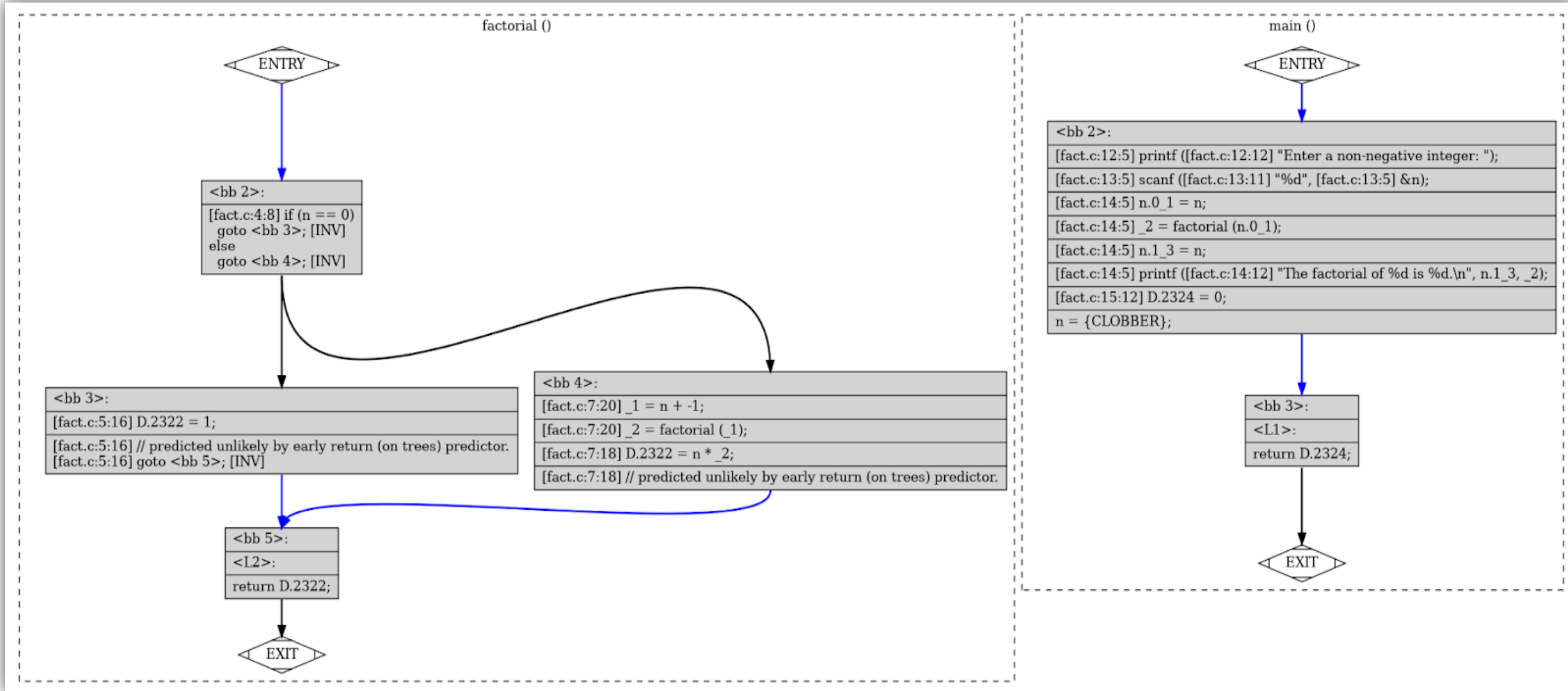
fact.c.012t.
cfg.dot

```
    printf("Enter a non-negative integer: ");
    scanf("%d", &n);
    printf("The factorial of %d is %d\n", n, factorial(n));
    return 0;
}
```

Step 2[Convert .dot to .png]

dot -Tpng fact.c.012t.cfg.dot -o fact.c.png

```
return 0;
}
```

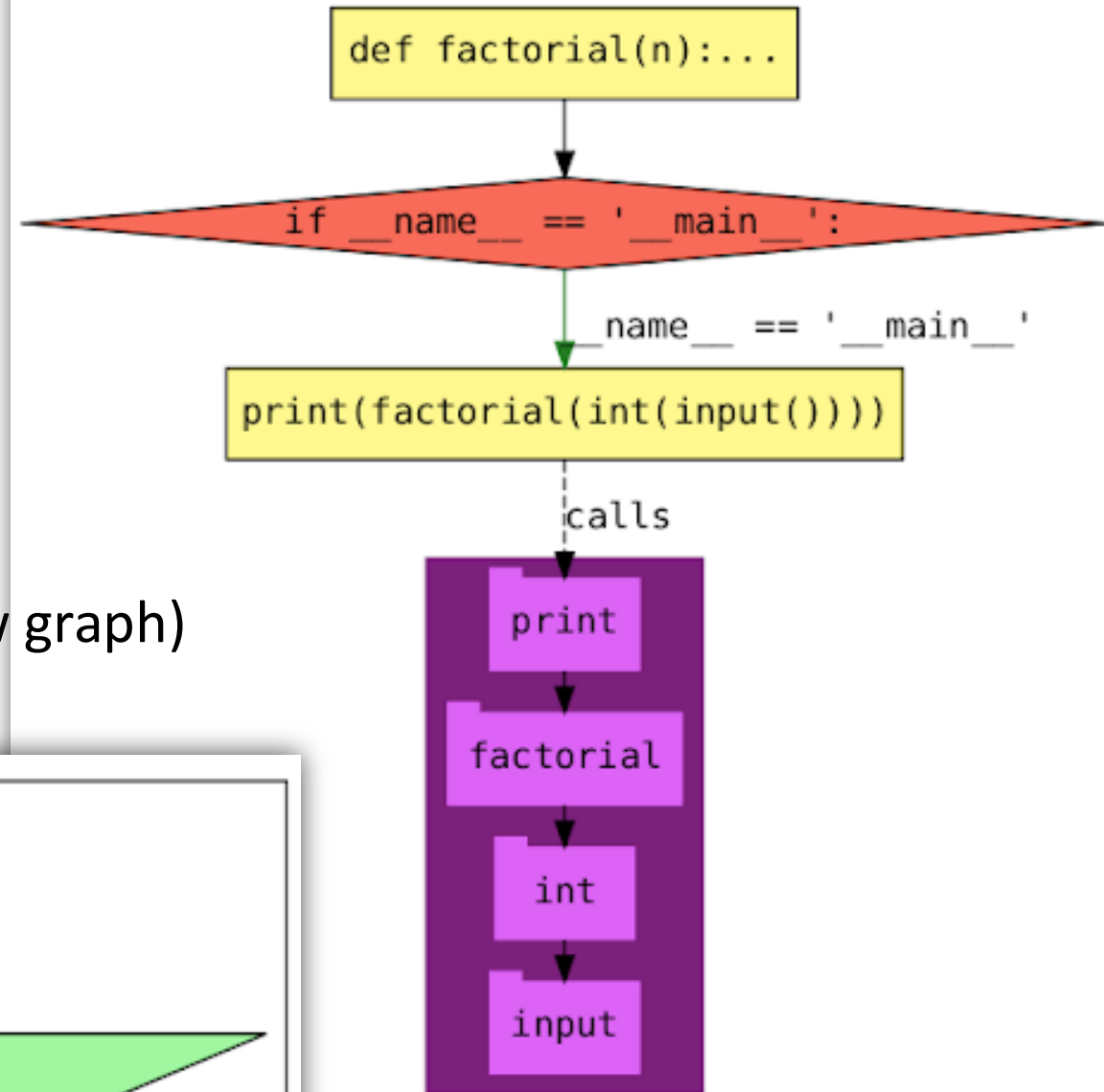
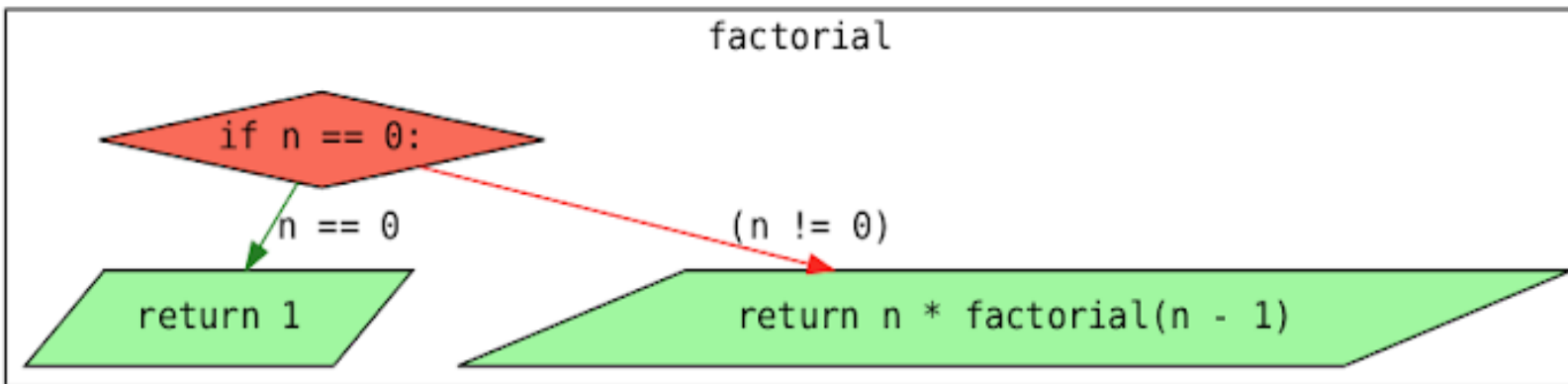


This is inter-procedural Control Flow Graph (CFG). What is 'intra'/'inter' here?

The Pythonic way...

```
#filename: fact.py
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)
if __name__ == "__main__":
    print(factorial(int(input())))
```

[**Graphical mode**] (generate extended control flow graph)
py2cfg fact.py



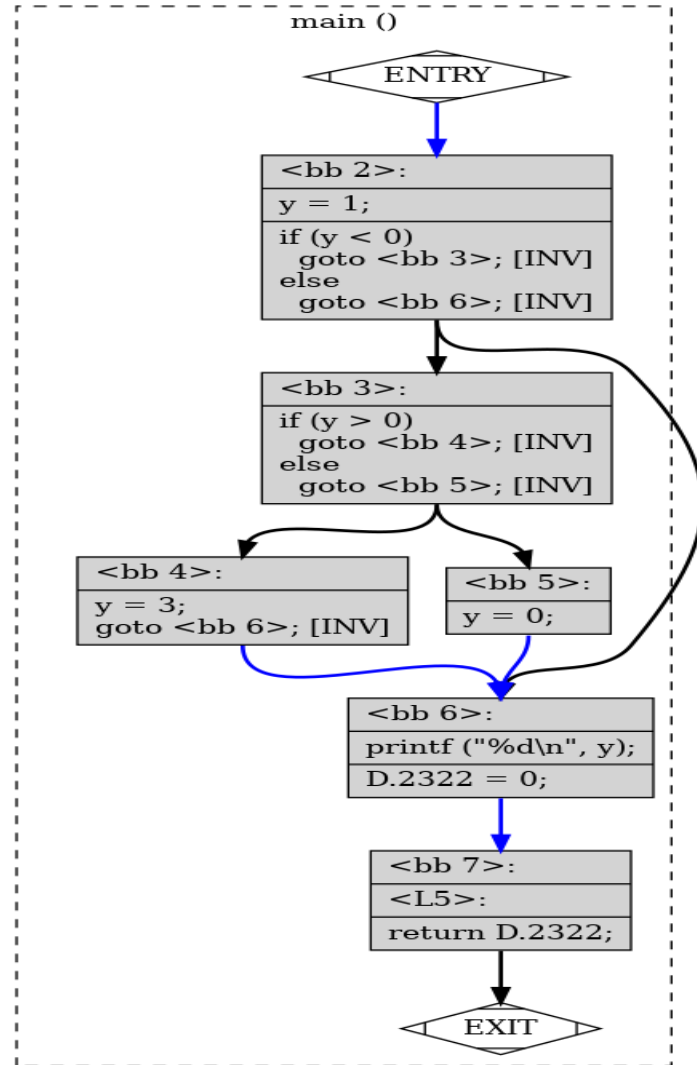
Classwork #1: Draw the CFG for...

C	Python
<pre>#include<stdio.h> int main() { int y=1; if(y<0) if (y>0) y=3; else y=0; printf("%d\n",y); }</pre>	<pre>if __name__ == "__main__": y=1; if(y<0): if (y>0): y=3; else: y=0; print(y);</pre>

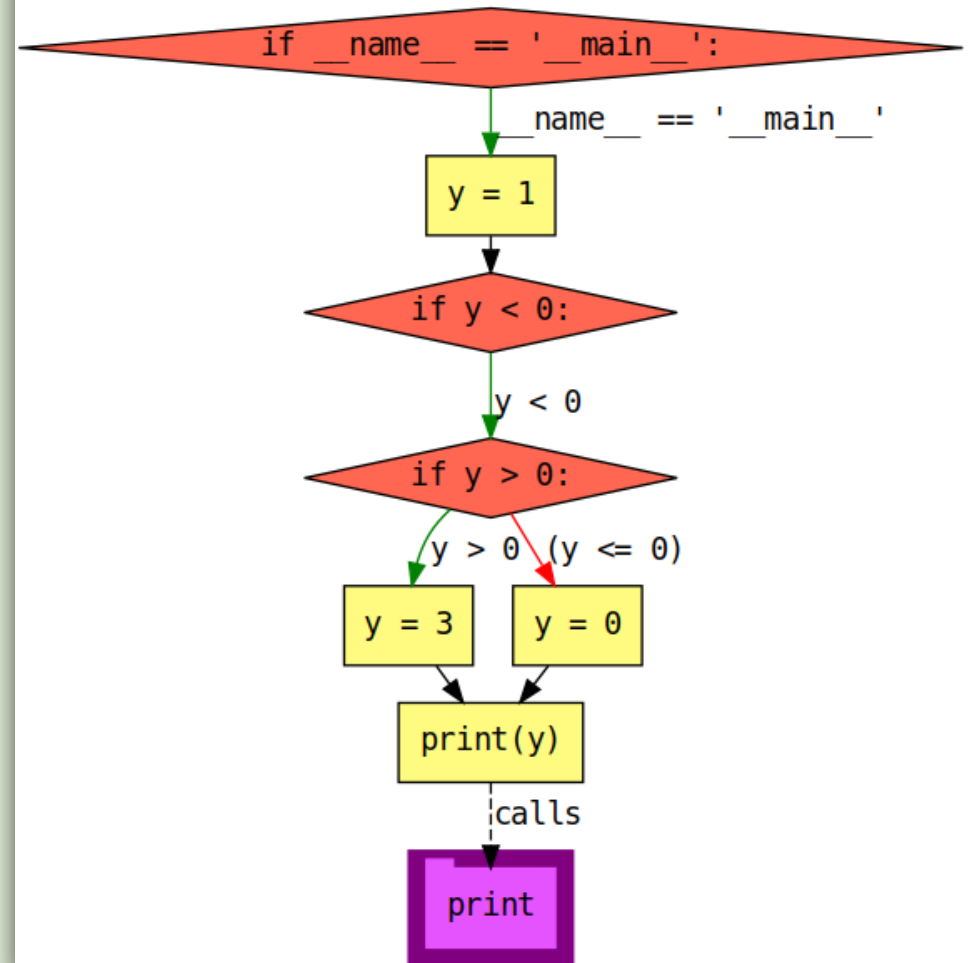
Same or different? If yes, why, If not, why not?

Classwork: Draw the CFG for the following...

C



Python

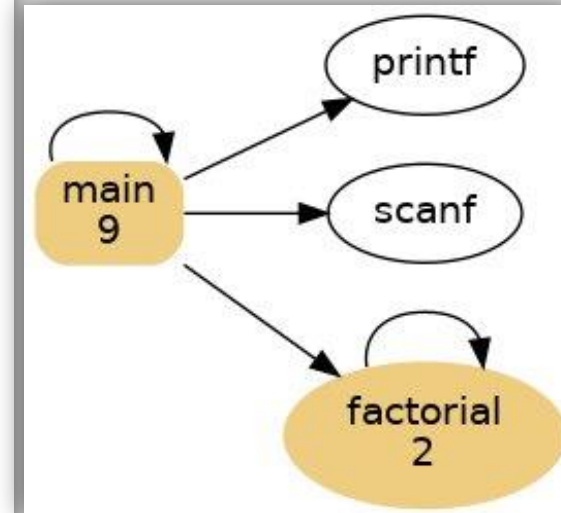
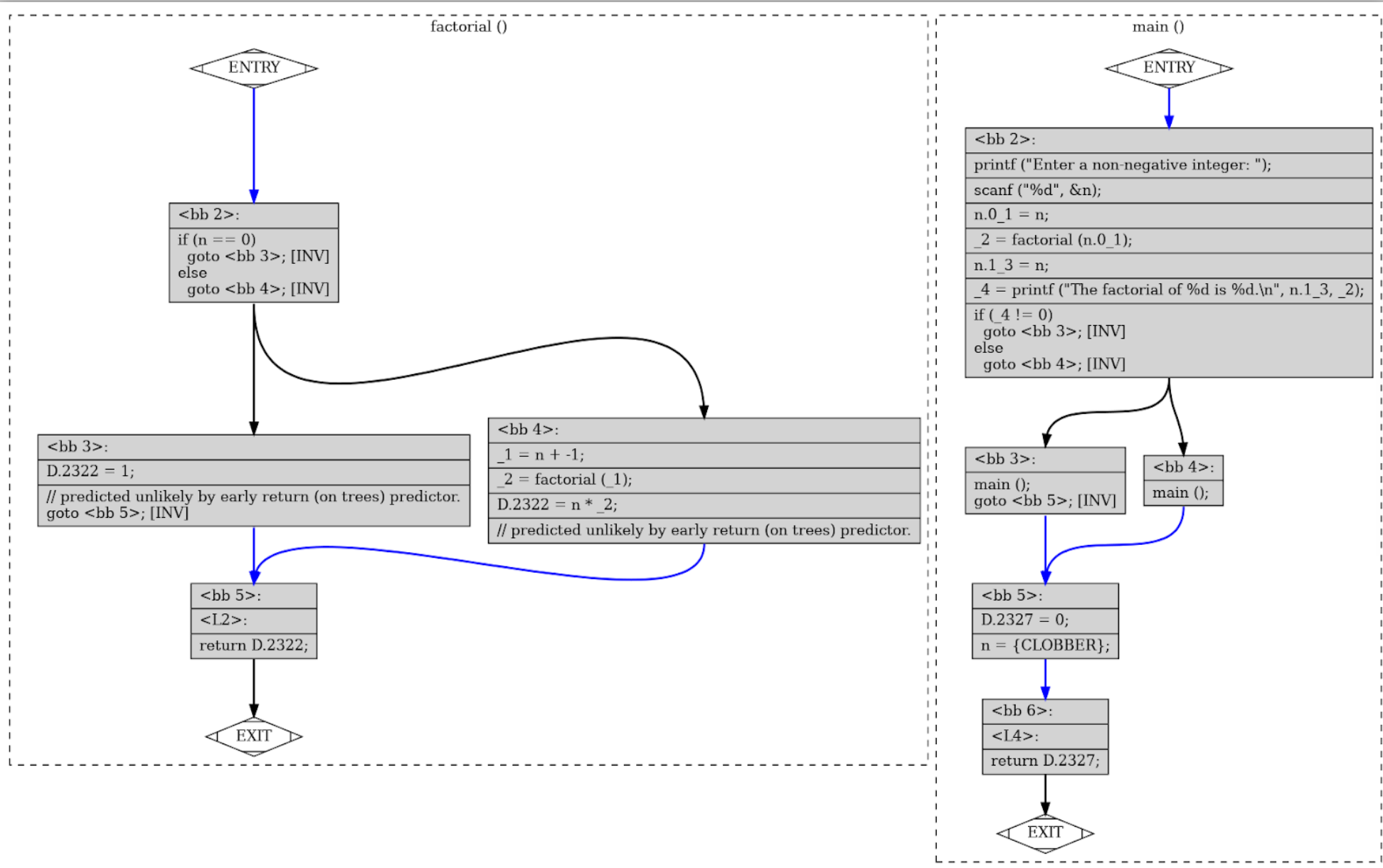


Classwork #2: Draw the CFG for...

```
#include <stdio.h>
int factorial(int n)
{
    if (n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}
int main()
{
    int n;
    printf("Enter a non-negative integer: ");
    scanf("%d", &n);
    if(printf("The factorial of %d is %d.\n", n, factorial(n)))
    {
        main();
    }
    else
        main();
    return 0;
}
```

What will be the call graph like?

Classwork #2: Draw the CFG for...



Classwork #3: Algebraically derive the cyclomatic complexity

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    if(n-1)
    {
        printf("%d\n", n-1);
        if(n-2)
        {
            printf("%d\n", n-2);
            if(n-3)
            {
                printf("%d\n", n-3);
                if(n-4)
                {
                    printf("%d\n", n-4);
                    if(n-5)
                    {
                        printf("%d\n", n-5);
                    }
                }
            }
        }
    }
    return(0);
}
```

Algebraic

Domain specific recurrence

$$\begin{aligned}cc(n) &= 1 + cc(n-1) \\ &= 1 + 1 + cc(n-2)\end{aligned}$$

...

$$\begin{aligned}&= 5 + cc(n-5) \\ &= 5 + 1 = \mathbf{6}\end{aligned}$$

Therefore,
 $cc(\text{main}) = 6$

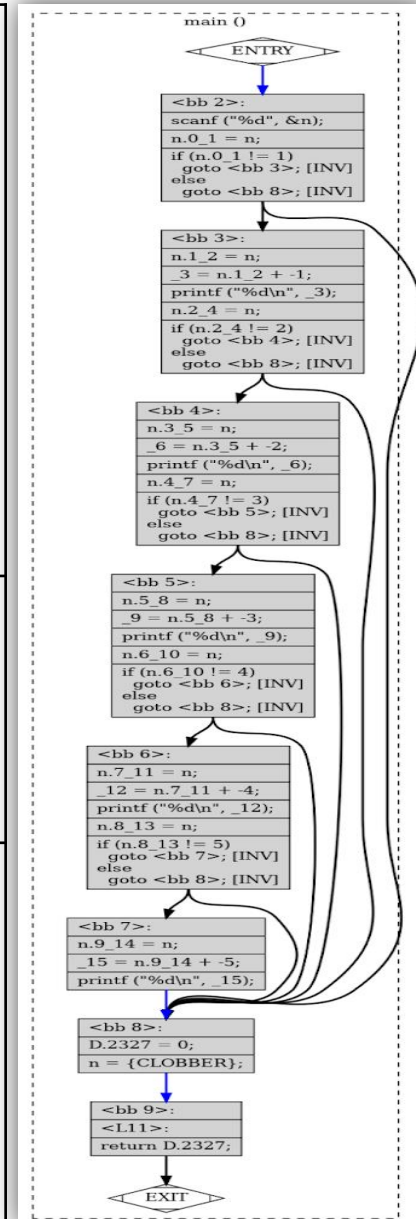
Sanity check #1

$$\begin{aligned}cc(\text{main}) &= \text{\#regions in CFG} \\ &= 5 + 1 = \mathbf{6}\end{aligned}$$

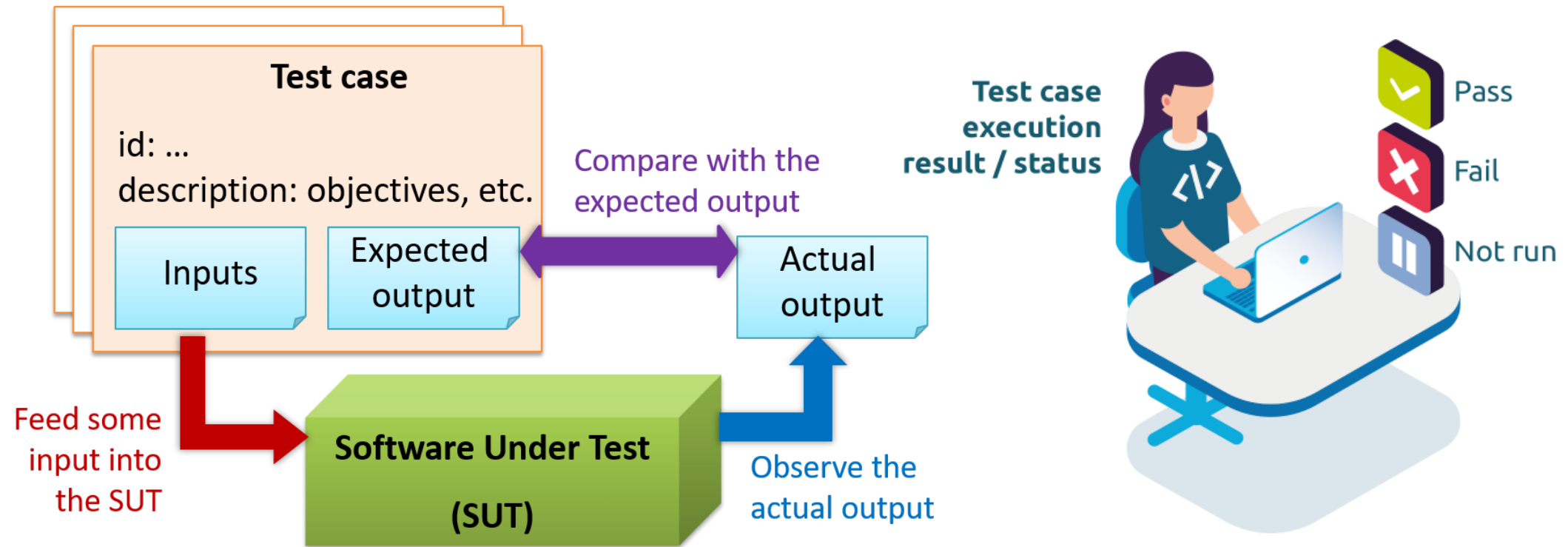
Sanity check #2

$$\begin{aligned}cc(\text{main}) &= \pi + 1 \\ &= 5 + 1 = \mathbf{6}\end{aligned}$$

π is the #decision nodes (here ifs)



How do we test for software bugs? Test cases



Test case: a scenario/use case/input which **executes** the software to *observe some interesting events*.

Example of testing-and-debugging a C program (for division): *version 0 with 4 test cases*

```
//original program
#include<stdio.h>
int main(void)
{
    int x,y;
    float res;
    scanf("%d %d",&x, &y);

    res=x/y;
    printf("%d/%d =%.2f\n",x,y,res);

    return (0);
}
```

```
//statement of interest
#include<stdio.h>
int main(void)
{
    int x,y;
    float res;
    scanf("%d %d",&x, &y);

    res=x/y;
    printf("%d/%d =%.2f\n",x,y,res);

    return (0);
}
```

Example of testing-and-debugging a C program (for division): *version 0 with 4 test cases (using gcc 9.3.0)*

```
//statement of interest
#include<stdio.h>
int main(void)
{
    int x,y;
    float res;
    scanf("%d %d",&x, &y);

    res=x/y;
    printf("%d/%d =%.2f\n",x,y,res);

    return (0);
}
```

\$./a.out < t1

1/1 = 1.00

\$./a.out < t2

5/4 = 1.00

\$./a.out < t3

3/4 = 0.00

\$./a.out < t4

15/5 = 3.00

Example of testing-and-debugging a C program (for division): *version 1 with 4 test cases (using gcc 9.3.0)*

```
//statement of interest
#include<stdio.h>
int main(void)
{
    int x,y;
    float res;
    scanf("%d %d",&x, &y);

    res=(float)(x/y);
    printf("%d/%d =%.2f\n",x,y,res);

    return (0);
}
```

\$./a.out < t1

1/1 = 1.00

\$./a.out < t2

5/4 = 1.00

\$./a.out < t3

3/4 = 0.00

\$./a.out < t4

15/5 = 3.00

Example of testing-and-debugging a C program (for division): *version 2 with 4 test cases (using gcc 9.3.0)*

```
//statement of interest
#include<stdio.h>
int main(void)
{
    int x,y;
    float res;
    scanf("%d %d",&x, &y);

    res=(float)x/y;
    printf("%d/%d =%.2f\n",x,y,res);

    return (0);
}
```

\$./a.out < t1

1/1 = 1.00

\$./a.out < t2

5/4 = 1.25

\$./a.out < t3

3/4 = 0.75

\$./a.out < t4

15/5 = 3.00

Example of testing-and-debugging a C program (for division): *version 2 with 7 test cases (using gcc 9.3.0)*

```
//statement of interest
#include<stdio.h>
int main(void)
{
    int x,y;
    float res;
    scanf("%d %d",&x, &y);

    res=(float)x/y;
    printf("%d/%d =%.2f\n",x,y,res);

    return (0);
}
```

```
$ ./a.out < t1
```

```
...
```

```
$ ./a.out < t5
```

```
1/0 = inf
```

```
$ ./a.out < t6
```

```
0/0 = -nan
```

```
$ ./a.out < t7
```

```
0/1 = 0.00
```

Dynamic Program Analysis

```
//filename: fact.c
#include <stdio.h>
int factorial(int n)
{
    if (n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}
int main()
{
    int n;
    printf("Enter a non-negative integer: ");
    scanf("%d", &n);
    printf("The factorial of %d is %d.\n", n, factorial(n));
    return 0;
}
```

Illustrate an execution of the C code where we obtain statement (line) coverage as: (i) 100%, and (ii) < 100%.

```
//filename: fact.c
#include <stdio.h>
int factorial(int n)
{
```

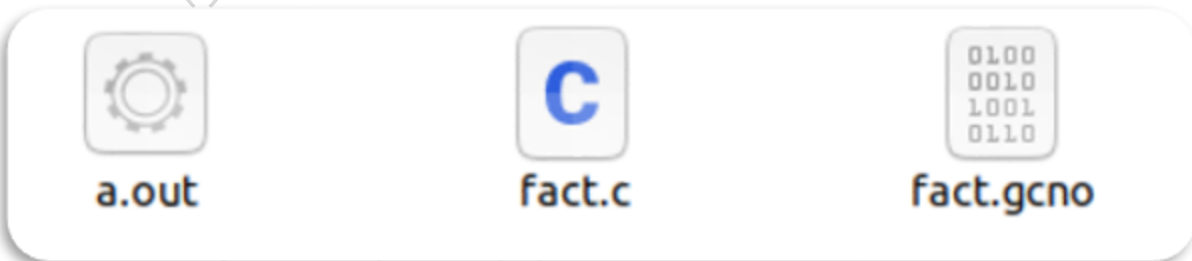


```
    return 1;
else
    return n * factorial(n - 1);
}
int main()
{
```

Step 1 [Instrumentation]:

gcc -Wall -fprofile-arcs -ftest-coverage fact.c

```
{
    printf("The factorial of %d is %d.\n", n, factorial(n));
    return 0;
}
```



```
printf("The factorial of %d is %d.\n", n, factorial(n));
return 0;
}
```

```
//filename: fact.c
```

```
#include <stdio.h>
```

```
int factorial(int n);
```

```
{
```



a.out



fact.c



fact.gcno

```
return 1;
```

Step 2 [Execution #1 of the instrumented a.out]:

`./a.out`

```
return n * factorial(n - 1);
```

```
}
```

```
int main()
```

```
{
```

Enter a non-negative integer: 5

The factorial of 5 is 120.



a.out



fact.c



fact.gcda



fact.gcno

```
return factorial(n));
```

```
}
```

```
//filename: fact.c
```

```
#include <stdio.h>
```

```
int  
{
```



a.out



fact.c



fact.gcda



fact.gcno

```
return 1;
```

Step3 [Show statement (line) coverage coverage]

gcov fact.c

```
return n * factorial(n - 1);  
}
```

```
int main()  
{
```

File 'fact.c'

Lines executed: 100.00% of 9

Creating 'fact.c.gcov'

```
printf("Enter a non-negative integer: ");
```

```
scanf("%d", &n);
```

```
printf("The factorial of %d is %d.\n", n, factorial(n));
```

```
return 0;
```

```
}
```

Step 4 [View fact.c.gcov]

```
-: 0:Source:fact.c
-: 0:Graph:fact.gcno
-: 0:Data:fact.gcda
-: 0:Runs:1
```

Shows statement (line) level #executions

```
-: 1:#include <stdio.h>
✓ 6: 2:int factorial(int n)
-: 3:{
✓ 6: 4:     if (n == 0)
✓ 1: 5:         return 1;
-: 6:     else
✓ 5: 7:         return n * factorial(n - 1);
-: 8:}
✓ 1: 9:int main()
-: 10:{
-: 11:     int n;
✓ 1: 12:     printf("Enter a non-negative integer: ");
✓ 1: 13:     scanf("%d", &n);
✓ 1: 14:     printf("The factorial of %d is %d.\n", n, factorial(n));
✓ 1: 15:     return 0;
-: 16:}
```


Step 5 [Cleanup results of previous execution]

```
rm a.out *.gcno *.gcov *.gcda
```

Step 6 [Re-create instrumented a.out]

```
gcc -Wall -fprofile-arcs -ftest-coverage fact.c
```

```
//filename: fact.c
```

```
#include <stdio.h>
```

```
int factorial(int n);
```

```
{
```



a.out



fact.c



fact.gcno

```
return 1;
```

Step 7 [Execution #2 of the instrumented a.out]:

`./a.out`

```
return n * factorial(n - 1);
```

```
}
```

```
int main()
```

```
{
```

Enter a non-negative integer: 0

The factorial of 0 is 1.



a.out



fact.c



fact.gcda



fact.gcno

```
return factorial(n));
```

```
}
```

```
//filename: fact.c
```

```
#include <stdio.h>
```

```
int  
{
```



a.out



fact.c



fact.gcda



fact.gcno

```
return 1;
```

Step 8 [Show statement (line) coverage coverage]

gcov fact.c

File 'fact.c'

Lines executed: 88.89% of 9

Creating 'fact.c.gcov'

```
scanf("%d", &n);
```

```
printf("The factorial of %d is %d.\n", n, factorial(n));
```

```
return 0;
```

```
}
```

Step 9 [View fact.c.gcov]

```
-: 0:Source:fact.c
-: 0:Graph:fact.gcno
-: 0:Data:fact.gcda
-: 0:Runs:1
```

Recursion (feature) is untested...

```
-: 1:#include <stdio.h>
```

```
1: 2:int factorial(int n)
```

```
-: 3:{
```

```
1: 4:     if (n == 0)
```

```
1: 5:         return 1;
```

```
-: 6:     else
```

```
✗ #####: 7:         return n * factorial(n - 1);
```

```
-: 8:}
```

```
1: 9:int main()
```

```
-: 10:{
```

```
-: 11:     int n;
```

```
1: 12:     printf("Enter a non-negative integer: ");
```

```
1: 13:     scanf("%d", &n);
```

```
1: 14:     printf("The factorial of %d is %d.\n", n, factorial(n));
```

```
1: 15:     return 0;
```

```
-: 16:}
```

Step 10 [Use `lcov + genhtml`] (auto-generating visualizations)

```
lcov --capture --directory . --output-file coverage.info  
genhtml coverage.info
```

Reading data file `coverage.info`

...

Overall coverage rate:

lines.....: 88.9% (8 of 9 lines)

functions...: 100.0% (2 of 2 functions)

LCOV - code coverage report

Current view: **top level**

Test: **coverage.info**

Date: **2024-01-09 00:10:32**

	Hit	Total	Coverage
Lines:	8	9	88.9 %
Functions:	2	2	100.0 %

Directory	Line Coverage 	Functions 
tut	88.9 % 8 / 9	100.0 % 2 / 2

Generated by: [LCOV version 1.14](#)

These are interactive deliverables

LCOV - code coverage report

Current view: [top level](#) - [tut](#) - [fact.c](#) ([source](#) / [functions](#))

Test: [coverage.info](#)

Date: [2024-01-09 00:10:32](#)

	Hit	Total	Coverage
Lines:	8	9	88.9 %
Functions:	2	2	100.0 %

Line data		Source code	<i>These are interactive deliverables</i>
1		: #include <stdio.h>	
2	1	: int factorial(int n)	
3		: {	
4	1	: if (n == 0)	
5	1	: return 1;	
6		: else	
7	0	: return n * factorial(n - 1);	
8		: }	
9	1	: int main()	
10		: {	
11		: int n;	
12	1	: printf("Enter a non-negative integer: ");	
13	1	: scanf("%d", &n);	
14	1	: printf("The factorial of %d is %d.\n", n, factorial(n));	
15	1	: return 0;	
16		: }	

Generated by: [LCOV version 1.14](#)

The Pythonic way...

```
#filename: fact.py
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)
if __name__ == "__main__":
    print(factorial(int(input())))
```

Illustrate an execution of the Python code where we obtain statement (line) coverage as:
(i) 100%, and (ii) < 100%.

coverage run fact.py

5

120

coverage html

Coverage report: 100%



coverage.py v7.4.0, created at 2024-01-09 00:47 +0530

Module	statements	missing	excluded	coverage
fact.py	6	0	0	100%
Total	6	0	0	100%

coverage.py v7.4.0, created at 2024-01-09 00:47 +0530

The Pythonic way...

```
#filename: fact.py
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)
if __name__ == "__main__":
    print(factorial(int(input())))
```

Illustrate an execution of the Python code where we obtain statement (line) coverage as:
(i) 100%, and (ii) < 100%.

coverage run fact.py

0

1

coverage html

Coverage for **fact.py**: 83%

6 statements 5 run 1 missing 0 excluded

« prev ^ index » next coverage.py v7.4.0, created at 2024-01-09 00:48 +0530

```
1 | def factorial(n):
2 |     if n == 0:
3 |         return 1
4 |     else:
5 |         return n * factorial(n-1)
6 |
7 | if __name__ == "__main__":
8 |     print(factorial(int(input())))
```


The Pythonic way...

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

Let us automatically generate some test cases...



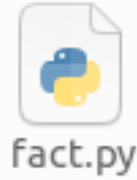
pynguin --project-path . --output-path . --module-name fact -v

```
[01:03:30] INFO      Start Pynguin Test Generation...  
INFO      Collecting static constants from module under test  
INFO      No constants found  
INFO      Setting up runtime collection of constants  
INFO      Analyzed project to create test cluster  
INFO      Modules:      1  
INFO      Functions:    1  
INFO      Classes:      11  
INFO      Using seed 1704742409855393620  
INFO      Using strategy: Algorithm.DYNAMOSA  
INFO      Instantiated 3 fitness functions  
INFO      Using CoverageArchive  
INFO      Using selection function: Selection.TOURNAMENT_SELECTION  
INFO      No stopping condition configured!  
INFO      Using fallback timeout of 600 seconds  
INFO      Using crossover function: SinglePointRelativeCrossOver  
INFO      Using ranking function: RankBasedPreferenceSorting  
INFO      Start generating test cases  
INFO      Initial Population, Coverage: 1.000000  
INFO      Algorithm stopped before using all resources.  
INFO      Stop generating test cases  
INFO      Start generating assertions  
INFO      Setup mutation controller  
INFO      Build AST for fact  
INFO      Mutate module fact  
generator.py:16  
generator.py:20  
generator.py:21  
generator.py:22  
module.py:131  
module.py:131  
module.py:132  
module.py:132  
generator.py:19  
generationalalgorithmfactory.py:30  
generationalalgorithmfactory.py:39  
generationalalgorithmfactory.py:34  
generationalalgorithmfactory.py:32  
generationalalgorithmfactory.py:11  
generationalalgorithmfactory.py:12  
generationalalgorithmfactory.py:33  
generationalalgorithmfactory.py:35  
generator.py:51  
searchobserver.py:7  
generator.py:52  
generator.py:52  
generator.py:59  
mutationadapter.py:7  
mutationadapter.py:6  
mutationadapter.py:6
```

The Pythonic way...

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

Let us automatically generate some test cases...



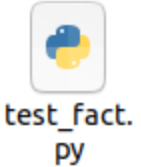
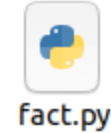
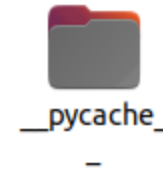
```
pynguin --project-path . --output-path . --module-name fact -v
```

```
INFO Mutate module fact mutationadapter.py:6
INFO Generated 9 mutants mutationadapter.py:7
INFO Running tests on mutant 1/9 assertiongenerator.py:29
INFO Running tests on mutant 2/9 assertiongenerator.py:29
INFO Running tests on mutant 3/9 assertiongenerator.py:29
INFO Running tests on mutant 4/9 assertiongenerator.py:29
INFO Running tests on mutant 5/9 assertiongenerator.py:29
INFO Running tests on mutant 6/9 assertiongenerator.py:29
INFO Running tests on mutant 7/9 assertiongenerator.py:29
INFO Running tests on mutant 8/9 assertiongenerator.py:29
INFO Running tests on mutant 9/9 assertiongenerator.py:29
INFO Mutant 0 killed by Test(s): 1 assertiongenerator.py:37
INFO Mutant 1 killed by Test(s): 1 assertiongenerator.py:37
INFO Mutant 2 killed by Test(s): 1 assertiongenerator.py:37
INFO Mutant 3 killed by Test(s): 0, 1 assertiongenerator.py:37
INFO Mutant 4 killed by Test(s): 1 assertiongenerator.py:37
INFO Mutant 5 killed by Test(s): 1 assertiongenerator.py:37
INFO Mutant 6 killed by Test(s): 0, 1 assertiongenerator.py:37
INFO Mutant 7 killed by Test(s): 0, 1 assertiongenerator.py:37
INFO Mutant 8 killed by Test(s): 1 assertiongenerator.py:37
INFO Number of Surviving Mutant(s): 0 (Mutants: ) assertiongenerator.py:38
INFO Calculating resulting FinalBranchCoverage generator.py:43
INFO Written 2 test cases to /home/shouvick/cache_test/tut/test_fact.py generator.py:70
INFO Writing statistics statistics.py:36
INFO Stop Pynquin Test Generation... generator.py:11
```

The Pythonic way...

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

Let us automatically generate some test cases...



pynguin --project-path . --output-path . --module-name fact -v

```
# Test cases automatically generated by Pynguin (https://www.pynguin.eu).  
# Please check them before you use them.  
import pytest  
import fact as module_0  
  
def test_case_0():  
    bool_0 = False  
    var_0 = module_0.factorial(bool_0)  
    assert var_0 == 1  
    var_1 = module_0.factorial(var_0)  
    assert var_1 == 1  
  
@pytest.mark.xfail(strict=True)  
def test_case_1():  
    list_0 = []  
    module_0.factorial(list_0)
```

Is this test case okay?

The Pythonic way...

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

Execute generated test cases from the same directory

pytest

```
import pytest  
import fact as module_0  
  
def test_case_0():  
    bool_0 = False  
    var_0 = module_0.factorial(bool_0)  
    assert var_0 == 1  
    var_1 = module_0.factorial(var_0)  
    assert var_1 == 1  
  
@pytest.mark.xfail(strict=True)  
def test_case_1():  
    list_0 = []  
    module_0.factorial(list_0)
```

```
===== test session starts =====  
platform linux -- Python 3.9.16, pytest-6.2.5, py-1.11.0, pluggy-1.3.0  
rootdir: /home/shouvick/demo/03_02_2025  
plugins: anyio-3.7.1, cov-4.1.0  
collected 2 items
```

```
test_fact.py .x [100%]  
===== 1 passed, 1 xfailed in 0.04s =====
```

The Pythonic way...

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

Execute generated test cases from the same directory

pytest

```
import pytest  
import fact as module_0
```

```
def test_case_1():  
    list_0 = []  
    module_0.factorial(list_0)
```

```
test_fact.py .F [100%]  
===== FAILURES =====  
----- test_case_1 -----
```

```
def test_case_1():  
    list_0 = []  
    > module_0.factorial(list_0)  
test_fact.py:18:
```

```
-----  
n = []
```

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        > return n * factorial(n-1)
```

```
E      TypeError: unsupported operand type(s) for -: 'list' and 'int'
```

```
fact.py:5: TypeError
```

```
===== short test summary info =====  
FAILED test_fact.py::test_case_1 - TypeError: unsupported operand type(s) for -: 'list' and 'int'  
===== 1 failed, 1 passed in 0.08s =====
```

Static versus Dynamic

- Sound
- Imprecise
- Input-oblivious
- Incomplete
- Precise
- Input-dependent

*Choosing between static and dynamic analysis often requires a trade-off between soundness and precision.
Current trend is to combine the two techniques to get better precision at improved scalability.*

Texts, References, and Acknowledgements

Online:

- Continuous Integration and Delivery (**CircleCI**: <https://circleci.com>)
- <http://www.cse.iitm.ac.in/~rupesh/teaching/pa/jan19>

Textbook:

- Sharp, J. (2022). *Microsoft Visual C# Step by Step*, 10th edition, Microsoft Press.
- Watson, K., Nagel, C., Pedersen, J. H., Reid, J. D., & Skinner, M. (2008). *Beginning Microsoft Visual C# 2008*. John Wiley & Sons.
- Mark J. Price (2024). *C# 13 and .NET 9 – Modern Cross-Platform Development Fundamentals*, 9th edition, Packt Publishing Ltd.

Reference:

- Soni, M. (2016). *DevOps for Web Development*. Packt Publishing Ltd.
- Yusuf Sulisty Nugroho, Hideaki Hata, and Kenichi Matsumoto. 2020. *How different are different diff algorithms in Git? Use --histogram for code changes*. Empirical Softw. Engg. 25, 1 (Jan 2020), 790–823.